GitHub Users API — Reference

API documentation for the two public GitHub endpoints used in the **GitHub Finder** exercise: fetching a user's profile and their repositories.

Base URL	`https://api.github.com`
Protocol	HTTPS only
Auth	None (public, unauthenticated access)
Format	JSON (`Content-Type: application/json; charset=utf-8`)
Methods used	`GET`

Authentication & Headers

No API key, token, or login is required for these endpoints.

No headers are required. Optionally, GitHub recommends sending:

Header	Value	Purpose
`Accept`	`application/vnd.github+json`	Pin the response media type
`User-Agent`	*your app name*	GitHub asks clients to identify themselves

---

Endpoint 1 — Get a user

Returns the public profile for a single user.

```
GET /users/{username}
```

Full URL

```
https://api.github.com/users/octocat
```

Path parameters

Name	Type	Required	Description
`username`	string	yes	The GitHub account handle (login), e.g. `octocat`

Query parameters

None.

Response `200 OK`

```
{
  "login": "octocat",
  "id": 583231,
  "avatar_url": "https://avatars.githubusercontent.com/u/583231?v=4",
  "html_url": "https://github.com/octocat",
  "name": "The Octocat",
  "company": "@github",
  "blog": "https://github.blog",
  "location": "San Francisco",
  "bio": null,
  "public_repos": 8,
  "public_gists": 8,
  "followers": 23132,
  "following": 9,
  "created_at": "2011-01-25T18:44:36Z",
  "updated_at": "2025-11-02T09:12:41Z"
}
```


Response fields

Field	Type	Description
`login`	string	The username / handle
`id`	number	Unique numeric user id
`avatar_url`	string (URL)	Profile picture
`html_url`	string (URL)	Link to the profile on github.com
`name`	string | null	Display name (can be `null`)
`company`	string | null	Company (can be `null`)
`blog`	string | null	Website URL (can be empty/`null`)
`location`	string | null	Location (can be `null`)
`bio`	string | null	Short bio (can be `null`)
`public_repos`	number	Count of public repositories
`public_gists`	number	Count of public gists
`followers`	number	Follower count
`following`	number	Following count
`created_at`	string (ISO 8601)	Account creation timestamp
`updated_at`	string (ISO 8601)	Last profile update timestamp

> Fields marked `null` may be absent of a value — always handle `null` on the client.

Endpoint 2 — List a user's repositories

Returns an **array** of the user's public repositories.

```
GET /users/{username}/repos
```

Full URL (with recommended params)

```
https://api.github.com/users/octocat/repos?sort=stars&per_page=5
```

Path parameters

Name	Type	Required	Description
`username`	string	yes	The GitHub account handle

Query parameters

Name	Type	Default	Description
`sort`	string	`full_name`	Order by: `created`, `updated`, `pushed`, `full_name` (use `stars` in this exercise to surface popular repos)
`direction`	string	`asc` / `desc`	Sort direction: `asc` or `desc`
`per_page`	number	`30`	Results per page (max `100`)
`page`	number	`1`	Page number for pagination
`type`	string	`owner`	Filter: `all`, `owner`, `member`

Response `200 OK`

An array of repository objects (one shown):


```
[
  {
    "id": 132935648,
    "name": "boysenberry-repo-1",
    "full_name": "octocat/boysenberry-repo-1",
    "html_url": "https://github.com/octocat/boysenberry-repo-1",
    "description": "Testing",
    "fork": false,
    "language": null,
    "stargazers_count": 455,
    "watchers_count": 455,
    "forks_count": 26,
    "open_issues_count": 12,
    "created_at": "2018-05-10T17:51:29Z",
    "updated_at": "2026-06-30T16:55:08Z"
  }
]
```


Response fields (per repository)

Field	Type	Description
`id`	number	Unique repository id
`name`	string	Repository name
`full_name`	string	`owner/name`
`html_url`	string (URL)	Link to the repo on github.com
`description`	string | null	Repo description (can be `null`)
`fork`	boolean	Whether the repo is a fork
`language`	string | null	Primary language (can be `null`)
`stargazers_count`	number	Number of stars
`watchers_count`	number	Number of watchers
`forks_count`	number	Number of forks
`open_issues_count`	number	Open issues + PRs
`created_at`	string (ISO 8601)	Creation timestamp
`updated_at`	string (ISO 8601)	Last update timestamp

Status codes

Code	Meaning	When
`200 OK`	Success	Request succeeded; body contains the data
`304 Not Modified`	Cached	Sent with conditional requests (ETag)
`403 Forbidden`	Rate limited	You exceeded the request limit (see below)
`404 Not Found`	Missing	No user exists with that username
`422 Unprocessable`	Invalid	Malformed request parameters

`404 Not Found` response body

```
{
  "message": "Not Found",
  "documentation_url": "https://docs.github.com/rest",
  "status": "404"
}
```

⚠️ Client note: a `GET` for a non-existent user returns HTTP **404** with a `200`-style JSON body shape above. Because `fetch()` does **not** reject on 4xx/5xx, always inspect `response.status` / `response.ok` before parsing.

`403` rate-limit response body

```
{
  "message": "API rate limit exceeded for <ip>.",
  "documentation_url": "https://docs.github.com/rest/overview/resources-in-the-rest-api#rate-limitin
}
```

Rate limiting

Access	Limit
Unauthenticated (by IP)	**60 requests / hour**
Authenticated (with token)	5,000 requests / hour

Every response includes rate-limit headers:

Header	Description
`x-ratelimit-limit`	Total requests allowed in the window
`x-ratelimit-remaining`	Requests left in the current window
`x-ratelimit-reset`	UNIX timestamp when the window resets

Example requests

cURL

```
curl https://api.github.com/users/octocat
curl "https://api.github.com/users/octocat/repos?sort=stars&per_page=5"
```

JavaScript (fetch)

```
const res = await fetch("https://api.github.com/users/octocat");
if (!res.ok) throw new Error(`HTTP ${res.status}`); // handle 404 / 403
const user = await res.json();
```

Source: GitHub REST API v3 — `https://docs.github.com/rest/users`. Sample values captured from live responses; counts and timestamps will differ.